

High Performance Parallel Computing

Homework 1

Liam Pohlmann, Alex Shaffer

September 15, 2026

1 Problem 1

Using the provided MatLab code, MatLab version R2025b was run on the `eng-partition` on a single node with a single single task and 8 CPUs per task. The maximum number of GFlops per core is about 80.151, which occurs at $N = 1704$.

```
#!/bin/bash
#SBATCH --account=26fa-cs598pa-eng      # This is valid for all students
#SBATCH --partition=eng-instruction    # <- replace with your partition
#SBATCH --time=01:00:00
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --mem=16G
#SBATCH --job-name=run_matlab
#SBATCH --output=logfile

module load matlab/R2025b

matlab -batch "dgtime"
```

This produced the results seen in Figure 1.

A naive first iteration was created, with the implementation of a matrix-matrix being:

```
#pragma once

#include <stdlib.h>

/**
 * @brief Computes the matrix-matrix product  $C=AB$ 
 *
 * @param A A matrix
 * @param A_rows Number of rows in A
 * @param A_cols Number of columns in A
```

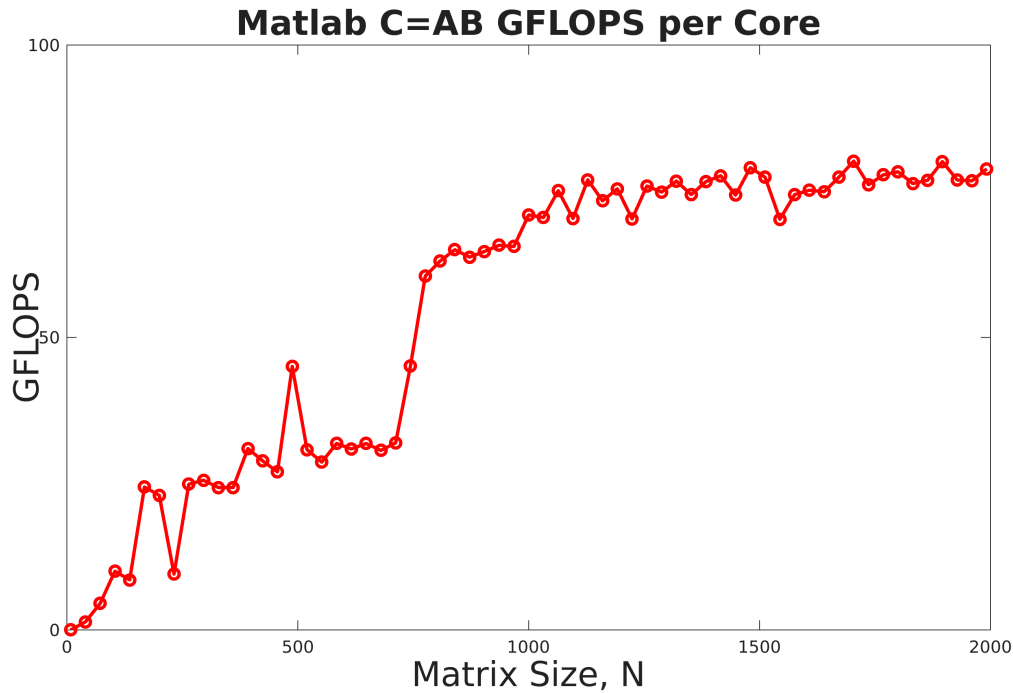


Figure 1: MatLab number of Giga-Flops per core for the matrix-matrix product.

```

* @param B B matrix
* @param B_rows Number of rows in B
* @param B_cols Number of columns in B
* @return double*
*/
double* MatMat(const double* A, const size_t A_rows, const size_t A_cols,
               const double* B, const size_t B_rows, const size_t B_cols);

/**
 * @brief Computes the i-j-th entry of the Matrix-Matrix product AB.
 *
 * @param A A matrix
 * @param A_cols Number of columns in A
 * @param B B matrix
 * @param B_cols Number of columns in B
 * @param row Row index (i)
 * @param col Column index (j)
 * @return double
 */
double MatMatEntry(const double* A, const size_t A_cols, const double* B,
                  const size_t B_cols, const size_t row, const size_t col);

/**
 * @brief Computes the 1D (flattened) index for an entry of a matrix. That is,
 * it converts a pair (i,j) to a single index k that indexes correctly into a 1D

```

```

* (flattened) representation of the matrix. This is computed with:
*
* k = n_cols * row + col
*
* @param n_cols Number of columns in matrix
* @param row Row index
* @param col Column index
* @return size_t
*/
size_t ComputeFlatIndex(const size_t n_cols, const size_t row,
                        const size_t col);

```

```

#include "mat_mat_multiply.h"

double* MatMat(const double* A, const size_t A_rows, const size_t A_cols,
               const double* B, const size_t B_rows, const size_t B_cols) {
    double* result = malloc(A_rows * B_cols * sizeof(double));
    for (size_t i = 0; i < A_rows; i++) {
        for (size_t j = 0; j < B_cols; j++) {
            size_t results_flat_index = ComputeFlatIndex(B_cols, i, j);
            result[results_flat_index] = MatMatEntry(A, A_cols, B, B_cols, i, j);
        }
    }
    return result;
}

double MatMatEntry(const double* A, const size_t A_cols, const double* B,
                  const size_t B_cols, const size_t row, const size_t col) {
    size_t K = A_cols;
    double sum = 0.0;
    for (size_t k = 0; k < K; k++) {
        size_t A_flat_index = ComputeFlatIndex(A_cols, row, k);
        size_t B_flat_index = ComputeFlatIndex(B_cols, k, col);
        sum += (A[A_flat_index] * B[B_flat_index]);
    }
    return sum;
}

size_t ComputeFlatIndex(const size_t n_cols, const size_t row,
                        const size_t col) {
    return n_cols * row + col;
}

```

with a Makefile:

```

CC = gcc
include ../common.mk

```

```

# ---- Assignment-specific settings ----
TARGET = hw1
SRCS    = $(wildcard src/*.c)
OBJS    = $(SRCS:.c=.o)

REPORT = report/report.tex

.PHONY: all code pdf clean run

all: code pdf

code: $(TARGET)

$(TARGET): $(OBJS)
    $(CC) $(OBJS) -o $@ $(LDFLAGS)

pdf:
    $(LATEXMK) $(LATEXOPTS) -cd $(REPORT)

run: code
    ./$$(TARGET)

clean:
    rm -f $(TARGET) $(OBJS)
    $(LATEXMK) -C -cd $(REPORT)

```

```

# common.mk
# Shared build settings included by every assignment's Makefile.
# Override any of these at the top of an assignment Makefile *before*
# the `include ../common.mk` line if a given assignment needs something
# different (e.g. CC = nvcc for a CUDA assignment).

# ---- C toolchain ----
ifeq ($(origin CC),default)
CC := mpicc
endif

CFLAGS    ?= -O2 -Wall -Wextra -std=c11 -fopenmp
LDFLAGS    ?= -fopenmp -lm

# ---- LaTeX toolchain ----
LATEXMK    ?= latexmk
LATEXOPTS  ?= -pdf -interaction=nonstopmode -halt-on-error -output-directory=build

# ---- Generic pattern rule for object files ----
# Assumes headers live in ./include relative to the assignment directory.
%.o: %.c

```

```
$(CC) $(CFLAGS) -Iinclude -c $< -o $@
```

The second (and all future iterations) were made using Claude Code (Sonnet 5.0). This produced the figure seen in Figure 2, which is an order of magnitude improvement but still off about a factor of five for what is desired.

```
#pragma once

#include <stdlib.h>

/**
 * @brief Computes the matrix-matrix product C=AB
 *
 * @param A A matrix
 * @param A_rows Number of rows in A
 * @param A_cols Number of columns in A
 * @param B B matrix
 * @param B_rows Number of rows in B
 * @param B_cols Number of columns in B
 * @return double*
 */
double* MatMat(const double* restrict A, const size_t A_rows,
               const size_t A_cols, const double* restrict B,
               const size_t B_rows, const size_t B_cols);

/**
 * @brief Computes the i-j-th entry of the Matrix-Matrix product AB.
 *
 * @param A A matrix
 * @param A_cols Number of columns in A
 * @param B B matrix
 * @param B_cols Number of columns in B
 * @param row Row index (i)
 * @param col Column index (j)
 * @return double
 */
double MatMatEntry(const double* A, const size_t A_cols, const double* B,
                  const size_t B_cols, const size_t row, const size_t col);

/**
 * @brief Computes the 1D (flattened) index for an entry of a matrix. That is,
 * it converts a pair (i,j) to a single index k that indexes correctly into a 1D
 * (flattened) representation of the matrix. This is computed with:
 *
 * k = n_cols * row + col
 *
 * @param n_cols Number of columns in matrix
 * @param row Row index

```

```

* @param col Column index
* @return size_t
*/
size_t ComputeFlatIndex(const size_t n_cols, const size_t row,
                        const size_t col);

```

```

#include "mat_mat_multiply.h"

#define TILE_ROWS 4
#define TILE_COLS 4

// Accumulates the TILE_ROWS x TILE_COLS block of C at (row0, col0) over the
// full K reduction in scalar registers, so each element of A is read once
// per k (reused across TILE_COLS columns) and each element of B is read
// once per k (reused across TILE_ROWS rows), instead of every C entry
// re-walking A and B from scratch with no reuse.
static void ComputeTile(const double* restrict A, size_t A_cols,
                        const double* restrict B, size_t B_cols,
                        double* restrict C, size_t row0, size_t col0,
                        size_t K) {
    const double* restrict a0 = A + (row0 + 0) * A_cols;
    const double* restrict a1 = A + (row0 + 1) * A_cols;
    const double* restrict a2 = A + (row0 + 2) * A_cols;
    const double* restrict a3 = A + (row0 + 3) * A_cols;

    double c00 = 0.0, c01 = 0.0, c02 = 0.0, c03 = 0.0;
    double c10 = 0.0, c11 = 0.0, c12 = 0.0, c13 = 0.0;
    double c20 = 0.0, c21 = 0.0, c22 = 0.0, c23 = 0.0;
    double c30 = 0.0, c31 = 0.0, c32 = 0.0, c33 = 0.0;

    for (size_t k = 0; k < K; k++) {
        const double* restrict b_row = B + k * B_cols + col0;
        const double b0 = b_row[0], b1 = b_row[1], b2 = b_row[2], b3 = b_row[3];

        double av = a0[k];
        c00 += av * b0;
        c01 += av * b1;
        c02 += av * b2;
        c03 += av * b3;

        av = a1[k];
        c10 += av * b0;
        c11 += av * b1;
        c12 += av * b2;
        c13 += av * b3;

        av = a2[k];
        c20 += av * b0;

```

```

    c21 += av * b1;
    c22 += av * b2;
    c23 += av * b3;

    av = a3[k];
    c30 += av * b0;
    c31 += av * b1;
    c32 += av * b2;
    c33 += av * b3;
}

double* restrict c0 = C + (row0 + 0) * B_cols + col0;
double* restrict c1 = C + (row0 + 1) * B_cols + col0;
double* restrict c2 = C + (row0 + 2) * B_cols + col0;
double* restrict c3 = C + (row0 + 3) * B_cols + col0;
c0[0] = c00; c0[1] = c01; c0[2] = c02; c0[3] = c03;
c1[0] = c10; c1[1] = c11; c1[2] = c12; c1[3] = c13;
c2[0] = c20; c2[1] = c21; c2[2] = c22; c2[3] = c23;
c3[0] = c30; c3[1] = c31; c3[2] = c32; c3[3] = c33;
}

// Scalar fallback for row/column remainders that don't fill a full
// TILE_ROWS x TILE_COLS tile.
static void ComputeBlockScalar(const double* restrict A, size_t A_cols,
                               const double* restrict B, size_t B_cols,
                               double* restrict C, size_t row_start,
                               size_t row_end, size_t col_start,
                               size_t col_end, size_t K) {
    for (size_t row = row_start; row < row_end; row++) {
        const double* restrict a_row = A + row * A_cols;
        double* restrict c_row = C + row * B_cols;
        for (size_t col = col_start; col < col_end; col++) {
            double sum = 0.0;
            for (size_t k = 0; k < K; k++) {
                sum += a_row[k] * B[k * B_cols + col];
            }
            c_row[col] = sum;
        }
    }
}

double* MatMat(const double* restrict A, const size_t A_rows,
               const size_t A_cols, const double* restrict B,
               const size_t B_rows, const size_t B_cols) {
    (void)B_rows;
    double* restrict C = malloc(A_rows * B_cols * sizeof(double));

    const size_t row_tiled_end = (A_rows / TILE_ROWS) * TILE_ROWS;

```

```

const size_t col_tiled_end = (B_cols / TILE_COLS) * TILE_COLS;

for (size_t row = 0; row < row_tiled_end; row += TILE_ROWS) {
    for (size_t col = 0; col < col_tiled_end; col += TILE_COLS) {
        ComputeTile(A, A_cols, B, B_cols, C, row, col, A_cols);
    }
    if (col_tiled_end < B_cols) {
        ComputeBlockScalar(A, A_cols, B, B_cols, C, row, row + TILE_ROWS,
                           col_tiled_end, B_cols, A_cols);
    }
}
if (row_tiled_end < A_rows) {
    ComputeBlockScalar(A, A_cols, B, B_cols, C, row_tiled_end, A_rows, 0,
                       B_cols, A_cols);
}

return C;
}

double MatMatEntry(const double* A, const size_t A_cols, const double* B,
                   const size_t B_cols, const size_t row, const size_t col) {
    size_t K = A_cols;
    double sum = 0.0;
    for (size_t k = 0; k < K; k++) {
        size_t A_flat_index = ComputeFlatIndex(A_cols, row, k);
        size_t B_flat_index = ComputeFlatIndex(B_cols, k, col);
        sum += (A[A_flat_index] * B[B_flat_index]);
    }
    return sum;
}

size_t ComputeFlatIndex(const size_t n_cols, const size_t row,
                        const size_t col) {
    return n_cols * row + col;
}

```

The final iteration is given in Figure 3 using the files:

```

#include "mat_mat_multiply.h"

// Register-resident accumulator tile shape (rows x cols), sized to the
// vector width -march=native actually enables so one row's accumulator
// fills whole vector registers instead of spilling or under-using them:
// AVX-512 packs 8 doubles/register, AVX2 packs 4. Matches the Makefile's
// -mprefer-vector-width choice for the same reason.
#ifdef __AVX512F__
#define TILE_ROWS 8
#define TILE_COLS 8

```

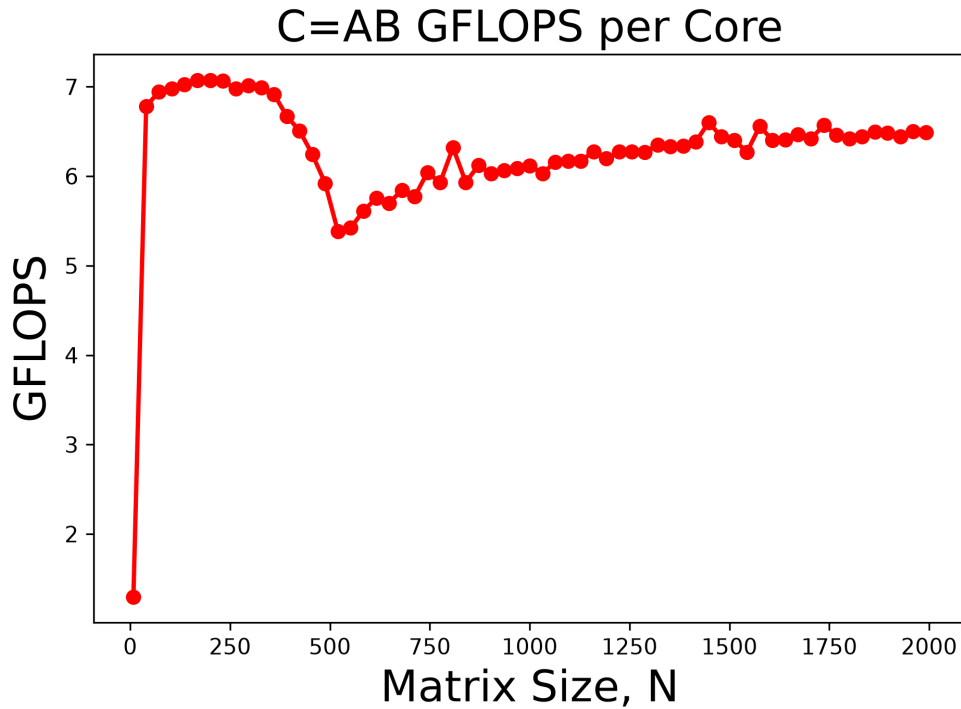


Figure 2: Second iteration.

```

#elif defined(__AVX2__) && defined(__FMA__)
#define TILE_ROWS 4
#define TILE_COLS 4
#else
#define TILE_ROWS 2
#define TILE_COLS 4
#endif

// Block sizes for the loop nest in MatMat. KC caps how many rows of B
// ([kc, kc+KC) x full width) are swept repeatedly while resident in
// cache/L3 -- in row-major storage that row-range of B is already one
// contiguous chunk of memory, so no packing copy is needed to get reuse
// out of it. MC caps how many rows of A (x KC cols) are pulled in per
// block so that block plus the KC-slab of B comfortably fit in cache
// alongside each other. Both are picked for a few-MB L2/L3 budget and are
// not re-derived from detected cache sizes, since -march=native isn't
// portable enough to assume a specific topology here.
#define KC 128
#define MC 96

static inline size_t Min(size_t a, size_t b) { return a < b ? a : b; }

// Accumulates the TILE_ROWS x TILE_COLS block of C at (row0, col0) over the

```

```

// reduction range [k0, k0+kdepth), in register-resident accumulators, so
// each element of A is read once per k (reused across TILE_COLS columns)
// and each element of B is read once per k (reused across TILE_ROWS rows).
// C is read once (existing partial sum from prior K-blocks) and written
// once per call, instead of every C entry re-walking A and B from scratch
// with no reuse. The inner loop is TILE_COLS wide to match one full vector
// register (see the TILE_COLS definition above); #pragma omp simd tells
// the vectorizer it may treat those TILE_COLS lanes as independent (each
// is its own reduction over k, so packing them into one vector op isn't a
// floating-point reordering of any single lane's sum).
static void ComputeTile(const double* restrict A, size_t A_cols,
                        const double* restrict B, size_t B_cols,
                        double* restrict C, size_t row0, size_t col0,
                        size_t k0, size_t kdepth) {
    const double* restrict a_row[TILE_ROWS];
    double* restrict c_row[TILE_ROWS];
    double acc[TILE_ROWS][TILE_COLS];
    for (size_t r = 0; r < TILE_ROWS; r++) {
        a_row[r] = A + (row0 + r) * A_cols + k0;
        c_row[r] = C + (row0 + r) * B_cols + col0;
        for (size_t c = 0; c < TILE_COLS; c++) acc[r][c] = c_row[r][c];
    }

    for (size_t k = 0; k < kdepth; k++) {
        const double* restrict b_row = B + (k0 + k) * B_cols + col0;
        for (size_t r = 0; r < TILE_ROWS; r++) {
            const double av = a_row[r][k];
#pragma omp simd
            for (size_t c = 0; c < TILE_COLS; c++) {
                acc[r][c] += av * b_row[c];
            }
        }
    }

    for (size_t r = 0; r < TILE_ROWS; r++)
        for (size_t c = 0; c < TILE_COLS; c++) c_row[r][c] = acc[r][c];
}

// Scalar fallback for row/column remainders that don't fill a full
// TILE_ROWS x TILE_COLS tile. Also accumulates into existing C values.
static void ComputeBlockScalar(const double* restrict A, size_t A_cols,
                               const double* restrict B, size_t B_cols,
                               double* restrict C, size_t row_start,
                               size_t row_end, size_t col_start,
                               size_t col_end, size_t k0, size_t kdepth) {
    for (size_t row = row_start; row < row_end; row++) {
        const double* restrict a_row = A + row * A_cols;
        double* restrict c_row = C + row * B_cols;

```

```

    for (size_t col = col_start; col < col_end; col++) {
        double sum = c_row[col];
        for (size_t k = 0; k < kdepth; k++) {
            sum += a_row[k0 + k] * B[(k0 + k) * B_cols + col];
        }
        c_row[col] = sum;
    }
}

// Runs the tiled microkernel (plus row/column-remainder fallback) over
// rows [row_start, row_end) x all of B_cols, for the reduction range
// [k0, k0+kdepth).
static void MultiplyRowBlock(const double* restrict A, size_t A_cols,
                             const double* restrict B, size_t B_cols,
                             double* restrict C, size_t row_start,
                             size_t row_end, size_t k0, size_t kdepth) {
    const size_t row_tiled_end =
        row_start + ((row_end - row_start) / TILE_ROWS) * TILE_ROWS;
    const size_t col_tiled_end = (B_cols / TILE_COLS) * TILE_COLS;

    for (size_t row = row_start; row < row_tiled_end; row += TILE_ROWS) {
        for (size_t col = 0; col < col_tiled_end; col += TILE_COLS) {
            ComputeTile(A, A_cols, B, B_cols, C, row, col, k0, kdepth);
        }
        if (col_tiled_end < B_cols) {
            ComputeBlockScalar(A, A_cols, B, B_cols, C, row, row + TILE_ROWS,
                               col_tiled_end, B_cols, k0, kdepth);
        }
    }
    if (row_tiled_end < row_end) {
        ComputeBlockScalar(A, A_cols, B, B_cols, C, row_tiled_end, row_end, 0,
                           B_cols, k0, kdepth);
    }
}

double* MatMat(const double* restrict A, const size_t A_rows,
               const size_t A_cols, const double* restrict B,
               const size_t B_rows, const size_t B_cols) {
    (void)B_rows;
    double* restrict C = calloc(A_rows * B_cols, sizeof(double));

    // K is blocked on the outside so a KC-row slab of B -- already
    // contiguous in row-major storage -- is swept by every row-block of A
    // while it's still warm in cache, instead of the full height of B being
    // re-streamed from memory once per row-block. M is then blocked so each
    // A/C block worked on within a slab stays cache-resident too.
    for (size_t k0 = 0; k0 < A_cols; k0 += KC) {

```

```

    const size_t kdepth = Min(KC, A_cols - k0);
    for (size_t row_start = 0; row_start < A_rows; row_start += MC) {
        const size_t row_end = Min(row_start + MC, A_rows);
        MultiplyRowBlock(A, A_cols, B, B_cols, C, row_start, row_end, k0,
                        kdepth);
    }
}

return C;
}

double MatMatEntry(const double* A, const size_t A_cols, const double* B,
                  const size_t B_cols, const size_t row, const size_t col) {
    size_t K = A_cols;
    double sum = 0.0;
    for (size_t k = 0; k < K; k++) {
        size_t A_flat_index = ComputeFlatIndex(A_cols, row, k);
        size_t B_flat_index = ComputeFlatIndex(B_cols, k, col);
        sum += (A[A_flat_index] * B[B_flat_index]);
    }
    return sum;
}

size_t ComputeFlatIndex(const size_t n_cols, const size_t row,
                       const size_t col) {
    return n_cols * row + col;
}

```

```

#pragma once

#include <stdlib.h>

/**
 * @brief Computes the matrix-matrix product C=AB
 *
 * @param A A matrix
 * @param A_rows Number of rows in A
 * @param A_cols Number of columns in A
 * @param B B matrix
 * @param B_rows Number of rows in B
 * @param B_cols Number of columns in B
 * @return double*
 */
double* MatMat(const double* restrict A, const size_t A_rows,
               const size_t A_cols, const double* restrict B,
               const size_t B_rows, const size_t B_cols);

/**

```

```

* @brief Computes the i-j-th entry of the Matrix-Matrix product AB.
*
* @param A A matrix
* @param A_cols Number of columns in A
* @param B B matrix
* @param B_cols Number of columns in B
* @param row Row index (i)
* @param col Column index (j)
* @return double
*/
double MatMatEntry(const double* A, const size_t A_cols, const double* B,
                  const size_t B_cols, const size_t row, const size_t col);

/**
* @brief Computes the 1D (flattened) index for an entry of a matrix. That is,
* it converts a pair (i,j) to a single index k that indexes correctly into a 1D
* (flattened) representation of the matrix. This is computed with:
*
* k = n_cols * row + col
*
* @param n_cols Number of columns in matrix
* @param row Row index
* @param col Column index
* @return size_t
*/
size_t ComputeFlatIndex(const size_t n_cols, const size_t row,
                      const size_t col);

```

```

CC = gcc
include ../common.mk

# ---- Performance overrides for the mat-mat multiply kernel ----
# -O3 : enable auto-vectorization of the blocked kernel
# -march=native : use the build host's SIMD ISA (AVX2/FMA, etc.)
# -ffp-contract=fast : fuse mul+add into FMA (off by default under -std=c11)
# -mprefer-vector-width=512 : let the auto-vectorizer use 512-bit (AVX-512)
# vectors on hosts that have them. mat_mat_multiply.c sizes its register
# tile from the same __AVX512F__/__AVX2__ feature macros -march=native
# defines, so the tile and the vector width always agree; on a host
# without AVX-512 this has no effect (there's nothing 512-bit to prefer).
# -fopenmp-simd : recognize the kernel's `#pragma omp simd` hints
# (needed so the vectorizer may reorder the tile's FP reduction -- IEEE754
# add isn't associative, so plain -O3 refuses to vectorize it otherwise)
# WITHOUT pulling in -fopenmp's threading runtime: it only enables SIMD
# directives, never `#pragma omp parallel`, and links no libgomp/pthread.
CFLAGS += -O3 -march=native -ffp-contract=fast -mprefer-vector-width=512 -fopenmp-simd

# Q1 requires single-core serial code. common.mk defaults CFLAGS/LDFLAGS to

```

```

# -fopenmp for other assignments in this repo; strip it here so nothing can
# quietly reintroduce multi-threading into this kernel.
# -fopenmp was also (incidentally) what made CLOCK_MONOTONIC visible under
# -std=c11 in main.c's tic()/toc(); replace that with an explicit POSIX
# feature-test macro now that -fopenmp is gone (see q2/Makefile for the
# same pattern).
CFLAGS := $(filter-out -fopenmp,$(CFLAGS)) -D_POSIX_C_SOURCE=199309L
LDFLAGS := $(filter-out -fopenmp,$(LDFLAGS))

# ---- Assignment-specific settings ----
TARGET = hw1
SRCS = $(wildcard src/*.c)
OBJS = $(SRCS:.c=.o)

REPORT = report/report.tex

.PHONY: all code pdf clean run

all: code pdf

code: $(TARGET)

$(TARGET): $(OBJS)
    $(CC) $(OBJS) -o $@ $(LDFLAGS)

pdf:
    $(LATEXMK) $(LATEXOPTS) -cd $(REPORT)

run: code
    ./$$(TARGET)

clean:
    rm -f $(TARGET) $(OBJS)
    $(LATEXMK) -C -cd $(REPORT)

```

```

#!/bin/bash
#SBATCH --account=26fa-cs598pa-eng      # This is valid for all students
#SBATCH --partition=eng-instruction    # <- replace with your partition
#SBATCH --time=01:00:00
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=16G
#SBATCH --job-name=run_hw1
#SBATCH --output=optimized_logfile

echo "=== compute node CPU ==="
lscpu | grep -Ei 'model name|^cpu\(s\)|mhz|cache'

```

```

echo "=== vector ISA available ==="
lscpu | grep -Eo 'avx512f|avx512vl|avx2|\bfma\b' | sort -u
echo "=== ISA gcc -march=native actually resolves to on this node ==="
gcc -march=native -Q --help=target 2>/dev/null | grep -E '\-m(avx|fma|sse|arch=| tune=)'

module load miniconda3/24.9.2

# `module load` only puts a bare base Python on PATH (no matplotlib), and
# `conda activate` needs conda.sh sourced before it works in a non-login
# shell. Create (once) and activate a dedicated env with the plotting deps,
# falling back to a user pip install if conda can't provision one.
source "$(conda info --base)/etc/profile.d/conda.sh"
CONDA_ENV=hw1-plot
if ! conda env list | grep -qE "^${CONDA_ENV}[:space:]"; then
    conda create -y -n "${CONDA_ENV}" python=3.11 matplotlib || true
fi
if conda env list | grep -qE "^${CONDA_ENV}[:space:]"; then
    conda activate "${CONDA_ENV}"
fi
python -c "import matplotlib" 2>/dev/null || python -m pip install --user --quiet matplotlib

make clean || true
make code
make run
python plot_gflops.py gflops.csv -o optimized.png
mv gflops.csv ./q1_results/
mv optimized.png ./q1_results/
mv optimized_logfile ./q1_results/

```

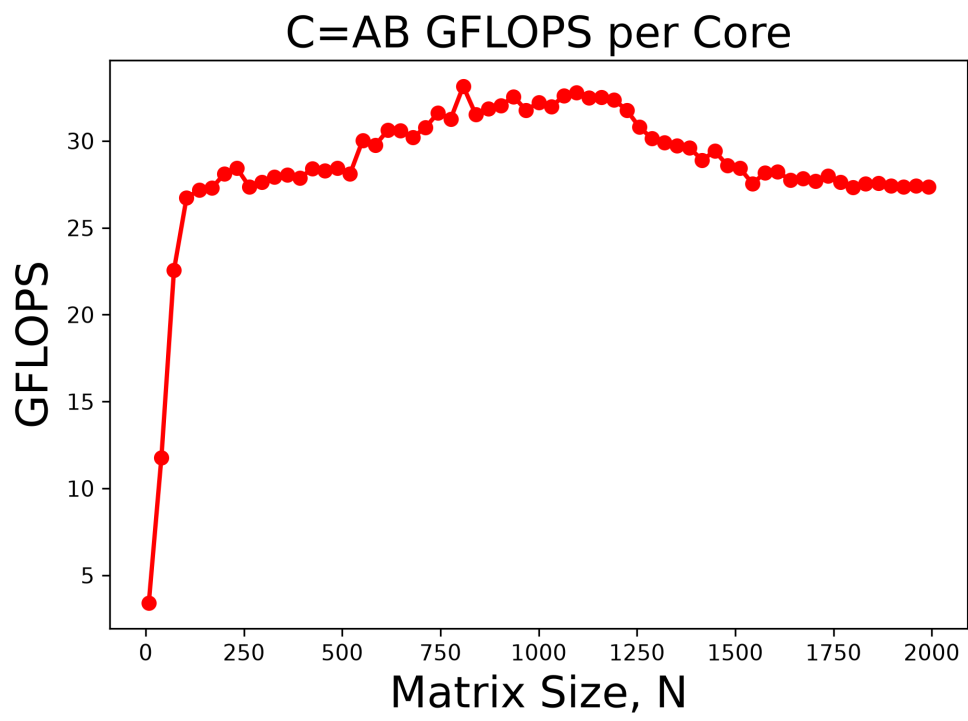


Figure 3: Last iteration.

Notice that with this final iteration, we were able to achieve about half of the performance of MatLab. This was only possible after giving Claude some additional information about the HPC that was being run on. This information was reported in the log file after a run and can be seen below.

```

=== compute node CPU ===
CPU(s):                                64
Model name:                            Intel(R) Xeon(R) Platinum 8358 CPU @ 2.60GHz
L1d cache:                             3 MiB (64 instances)
L1i cache:                             2 MiB (64 instances)
L2 cache:                              80 MiB (64 instances)
L3 cache:                              96 MiB (2 instances)
=== vector ISA available ===
avx2
avx512f
avx512vl
fma
=== ISA gcc -march=native actually resolves to on this node ===
-march=                                icelake-server
-mavx                                  [enabled]
-mavx2                                [enabled]
-mavx256-split-unaligned-load         [disabled]
-mavx256-split-unaligned-store        [disabled]
-mavx512fmaps                          [disabled]
-mavx512vnniw                          [disabled]
-mavx512bf16                          [disabled]
-mavx512bitalg                         [enabled]
-mavx512bw                             [enabled]
-mavx512cd                             [enabled]
-mavx512dq                             [enabled]
-mavx512er                             [disabled]
-mavx512f                              [enabled]
-mavx512ifma                          [enabled]
-mavx512pf                             [disabled]
-mavx512vbmi                          [enabled]
-mavx512vbmi2                         [enabled]
-mavx512vl                             [enabled]
-mavx512vnni                          [enabled]
-mavx512vp2intersect                  [disabled]
-mavx512vpopcntdq                     [enabled]
-mavxvnni                             [disabled]
-mfma                                  [enabled]
-mfma4                                 [disabled]
-msse                                  [enabled]
-msse2                                 [enabled]
-msse2avx                             [disabled]
-msse3                                 [enabled]
-msse4                                 [enabled]

```

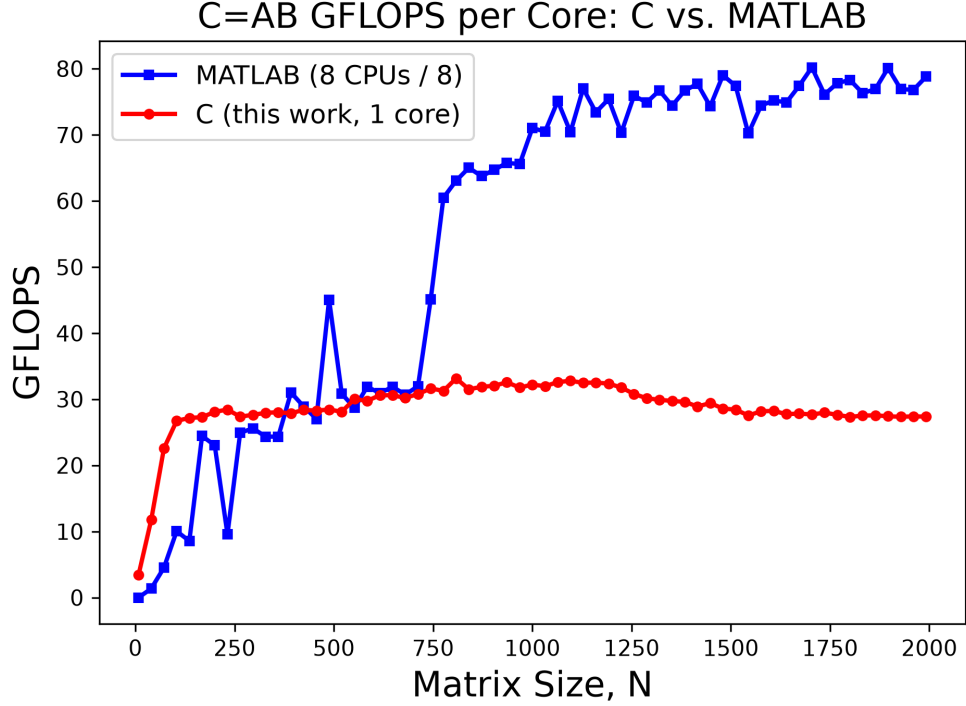


Figure 4: Final C kernel (1 core) vs. MatLab (8 CPUs / 8) GFLOPS per core.

`-msse4.1`

`[enabled]`

Raising `-mprefer-vector-width` to 512 and sizing the register tile in `mat_mat_multiply.c` off the same `__AVX512F__`/`__AVX2__` feature macros that `-march=native` defines (so the tile always matches the vector width actually available, and the code still compiles correctly on a partition node without AVX-512) took the achieved performance from about 7 GFLOPS/core up to a peak of 33.149 GFLOPS/core at $N = 808$, shown alongside the MatLab curve in Figure 4.

1.1 $N_{1/2}$

$N_{1/2}$ – the smallest matrix size at which a curve reaches half of its own peak GFLOPS – differs sharply between the two:

- **C (this work):** peak 33.149 GFLOPS/core; half-peak is 16.575 GFLOPS, first reached at $N_{1/2} = 72$.
- **MatLab:** peak 80.151 GFLOPS/core; half-peak is 40.076 GFLOPS, first reached at $N_{1/2} = 488$.

The serial C kernel has essentially no dispatch overhead, so it is already near its (lower) asymptotic rate at very small N . MatLab’s curve in Figure 4 is visibly a step function: below roughly $N = 700$ it tracks the C curve or falls under it, then jumps abruptly once N is large enough that MKL’s internal multithreading pays for itself, climbing toward its 8-CPU peak. That threading start-up cost is exactly why MatLab needs a much larger N to reach half of its own peak, even though its peak is higher.

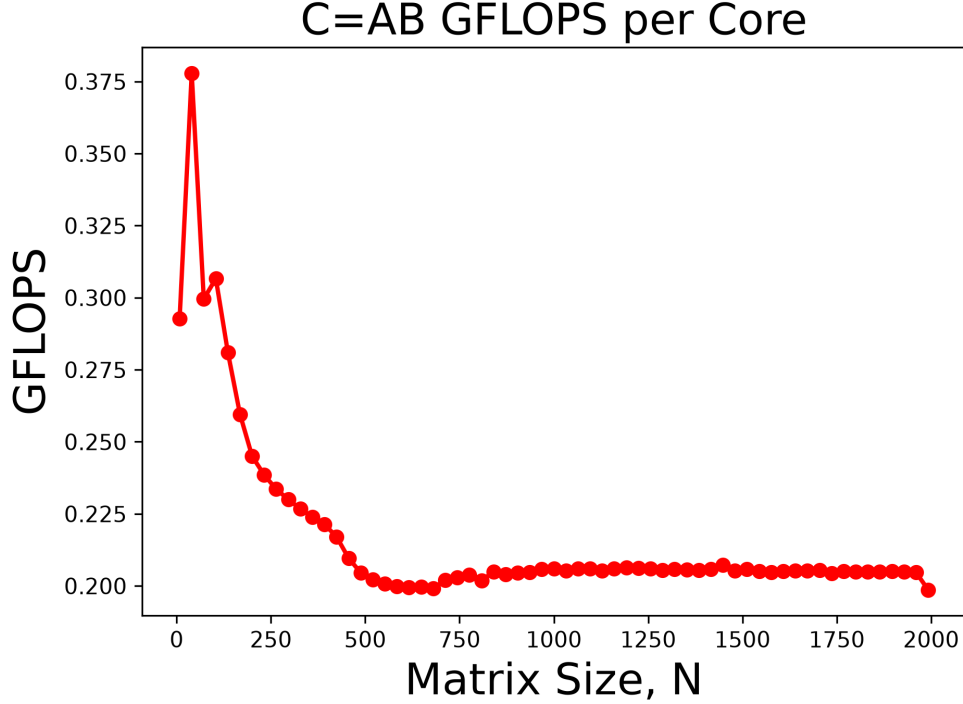


Figure 5: Iteration 1 (naive triple loop) GFLOPS per core.

1.2 Comparison to Iteration 1

Iteration 1 (the naive triple loop, Figure 5) peaked at only 0.378 GFLOPS/core (at $N = 40$) and settled to a floor of about 0.199 GFLOPS/core for large N , since its innermost loop strides through B with stride `B_cols` instead of reading it contiguously. Against that baseline, the final version is:

- $33.149/0.378 \approx 88\times$ faster at each curve’s own peak.
- $27.372/0.199 \approx 138\times$ faster at $N = 1992$, comparing large-matrix performance directly.

1.3 Process and Lessons Learned

The largest single-step gains, in order, came from: (1) loop reordering plus a register-blocked microkernel to stop re-walking A and B from scratch for every output entry, (2) blocking the K and M loops so a slab of B – already contiguous in row-major storage – stays cache-resident across many row-blocks of A instead of being re-streamed from memory each time, and (3) matching the microkernel’s vector width to what the compute node’s CPU actually supports.

Step (2) is a cautionary tale about environment mismatch: it produced a large, clean improvement on a local development machine (eliminating a severe falloff at large N), but made *no measurable difference at all* on the actual cluster node – the peak GFLOPS was identical before and after, and unaffected by changing `--cpus-per-task` from 1 to 8 and back. That flat ceiling was the signal that the bottleneck on the real hardware wasn’t memory bandwidth at all. Rather than keep guessing, we added a short diagnostic block to `batch_q1.sh` to print the compute node’s CPU model, advertised vector ISA, and what `-march=native` actually resolves to – which is what

surfaced step (3): the node is an Ice Lake Xeon with full AVX-512, but the Makefile was hard-capping vectorization at 256 bits and the microkernel tile was only sized for one 256-bit register per row, together leaving roughly half of the chip’s per-core throughput unused. That single fix accounted for most of the total improvement. The broader lesson: a local machine is not a reliable proxy for the target HPC hardware, and it’s worth pulling concrete diagnostics from the actual compute node early rather than assuming the same bottleneck applies everywhere.

We are satisfied with the final result. A peak of 33.149 GFLOPS/core against MatLab’s 80.151 is a real gap, but part of that gap is not apples-to-apples: MatLab’s “per-core” figure is an 8-CPU MKL run divided by 8, not a directly measured single-core run, and the plateau in Figure 4 strongly suggests it depends on multithreading to reach its peak at all. Closing more of the remaining gap on the C side would mean packing A/B into aligned contiguous panels and retuning the K/M block sizes for the Ice Lake node’s actual cache sizes rather than the development machine’s, but the current result is a reasonable, well-understood stopping point for a serial kernel.

2 Problem 2

Here we created the ping pong test to measure properties of the communication on one node and between 2 nodes. The ping pong test was run on the **eng-partition** of the campus cluster.

One note: though the assignment didn’t ask us to measure this, it became clear that the inter/intra socket communication on one node has a significant effect on the results. for this reason the plots are also broken up by socket.

2a.

Here we look at single node communication performance. We record the latency, inverse bandwidth, m_2 and eager limit. These are recorded for $P = 64$ in Table 1 and plotted in Figure 6. As noted before, the communication performance is measurably different depending on whether the communication is intra-socket or inter-socket. The eager limit is measured the same across these.

group	ranks	latency (μ s)	inv. bw (ns/word)	m_2 (words)	eager limit (bytes)
self/s0	1	1.281	1.0578	33	8088–8224
intra/s0	31	1.109	1.0583	23	8088–8224
intra/s1	32	1.013	0.7452	23	8088–8224

Table 1: Ping-pong summary, $P = 64$ on one node: median over the ranks in each group (self, same node, across nodes, by partner socket).

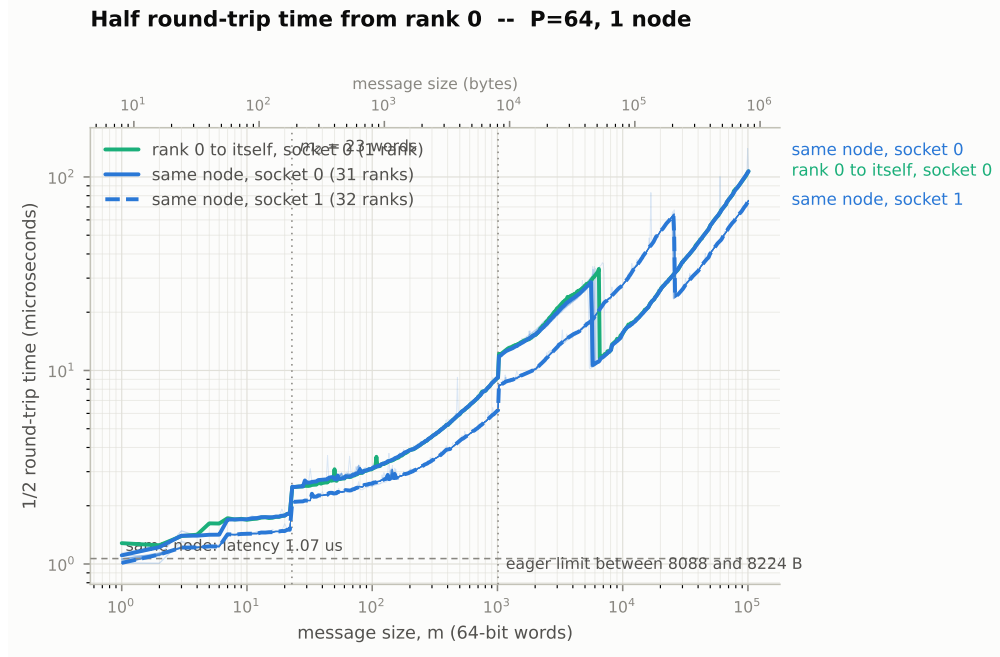


Figure 6: Half round-trip time vs. message size from rank 0 to every rank p , $P = 64$ on one node. Thin lines are individual partner ranks; heavy lines are the per-socket medians.

2b.

We move to $P = 128$ in Table 2 and Figure 7. The same trends are observed, but the latency is slightly higher for non-self communication.

group	ranks	latency (μ s)	inv. bw (ns/word)	m_2 (words)	eager limit (bytes)
self/s0	1	1.265	1.0641	23	8088–8224
intra/s0	63	1.141	1.0611	23	8088–8224
intra/s1	64	1.044	0.7518	23	8088–8224

Table 2: Ping-pong summary, $P = 128$ on one node: median over the ranks in each group (self, same node, across nodes, by partner socket).

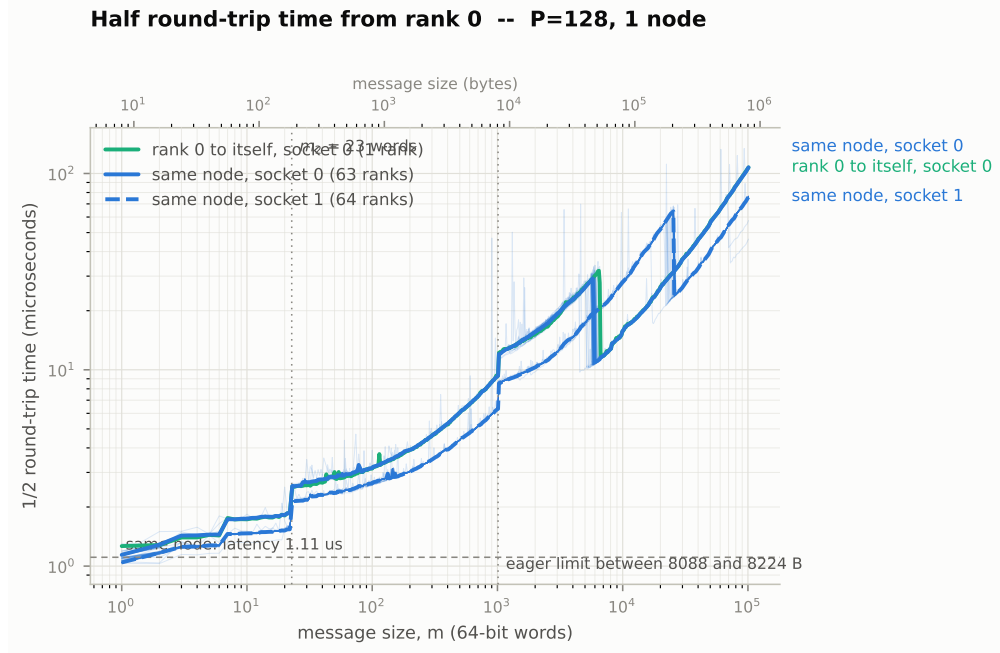


Figure 7: As Figure 6, $P = 128$ on one node.

We then move to $P = 128$ on two nodes in Table 3 and Figure 8. We see higher latency for the inter-node communication, the inter-node inverse bandwidth was measured to depend suprisingly on the socket. Socket S1 on node 2 was suprisingly close to the Socket 1 on node 1 for inverse bandwidth, but Socket 0 on node 2 was significantly worse. My guess is that this is due to the physical network connections on the cluster. The m_2 value is strikingly higher for the inter-node communication and again depends on the socket. The eager limit is measured to be the same across all sockets and nodes.

As we continue to increase in ranks on 2 nodes up to $P = 196$ and $P = 256$ we see the same trends.

group	ranks	latency (μ s)	inv. bw (ns/word)	m_2 (words)	eager limit (bytes)
self/s0	1	1.279	1.0554	32	8088–8224
intra/s0	31	1.114	1.0548	23	8088–8224
intra/s1	32	1.017	0.7466	23	8088–8224
inter/s0	32	2.663	3.0020	196	8088–8224
inter/s1	32	2.695	0.7904	410	8088–8224

Table 3: Ping-pong summary, $P = 128$ on two nodes: median over the ranks in each group (self, same node, across nodes, by partner socket).

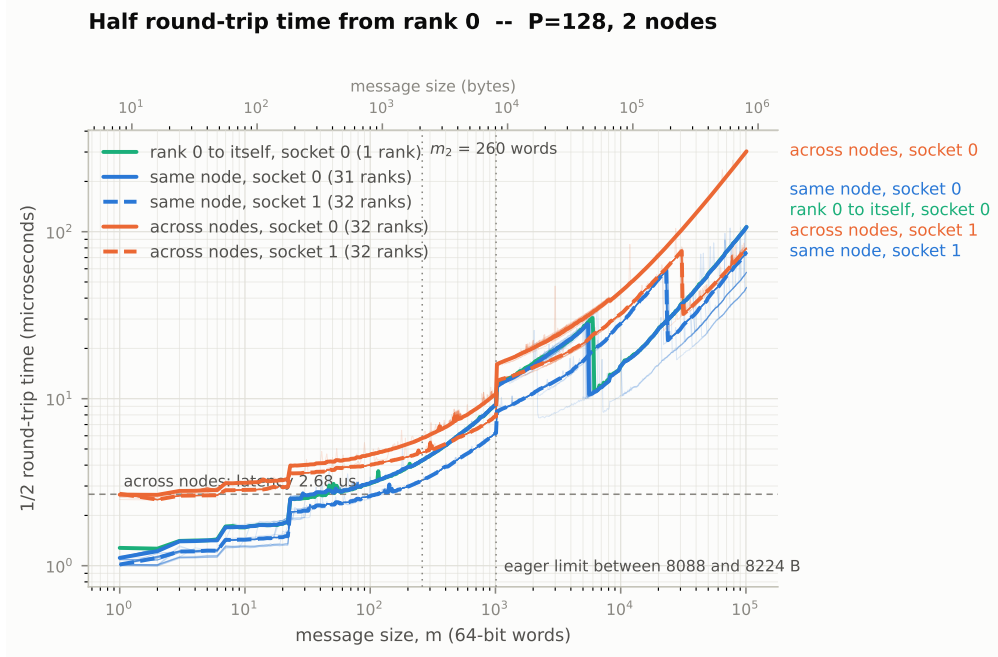


Figure 8: As Figure 6, $P = 128$ on two nodes.

group	ranks	latency (μ s)	inv. bw (ns/word)	m_2 (words)	eager limit (bytes)
self/s0	1	1.269	1.0598	28	8088–8224
intra/s0	47	1.124	1.0595	23	8088–8224
intra/s1	48	1.028	0.7533	23	8088–8224
inter/s0	48	2.754	3.0020	218	8088–8224
inter/s1	48	2.685	0.7901	400	8088–8224

Table 4: Ping-pong summary, $P = 192$ on two nodes: median over the ranks in each group (self, same node, across nodes, by partner socket).

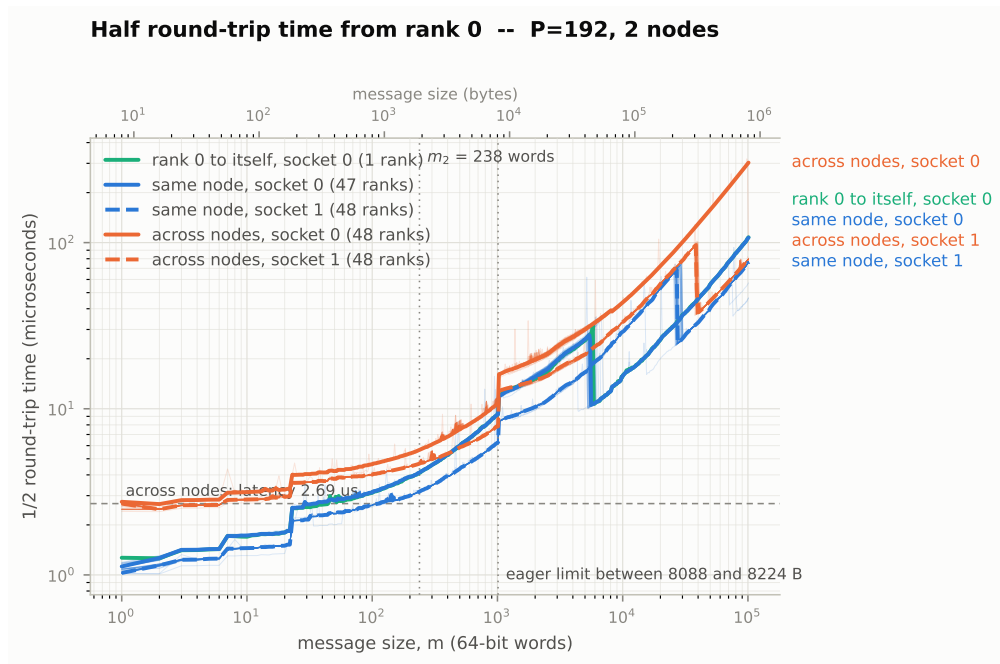


Figure 9: As Figure 6, $P = 192$ on two nodes.

group	ranks	latency (μ s)	inv. bw (ns/word)	m_2 (words)	eager limit (bytes)
self/s0	1	1.302	1.0702	33	8088–8224
intra/s0	63	1.156	1.0636	23	8088–8224
intra/s1	64	1.059	0.7545	23	8088–8224
inter/s0	64	2.790	3.0064	225	8088–8224
inter/s1	64	2.515	0.7931	304	8088–8224

Table 5: Ping-pong summary, $P = 256$ on two nodes: median over the ranks in each group (self, same node, across nodes, by partner socket).

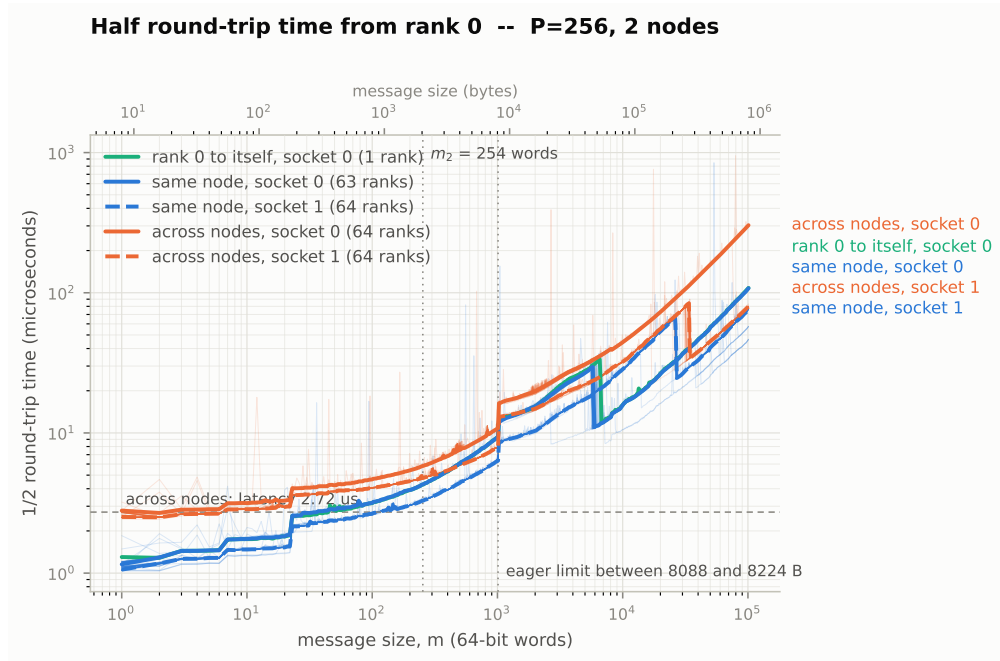


Figure 10: As Figure 6, $P = 256$ on two nodes.

Ping-pong scaling with rank count

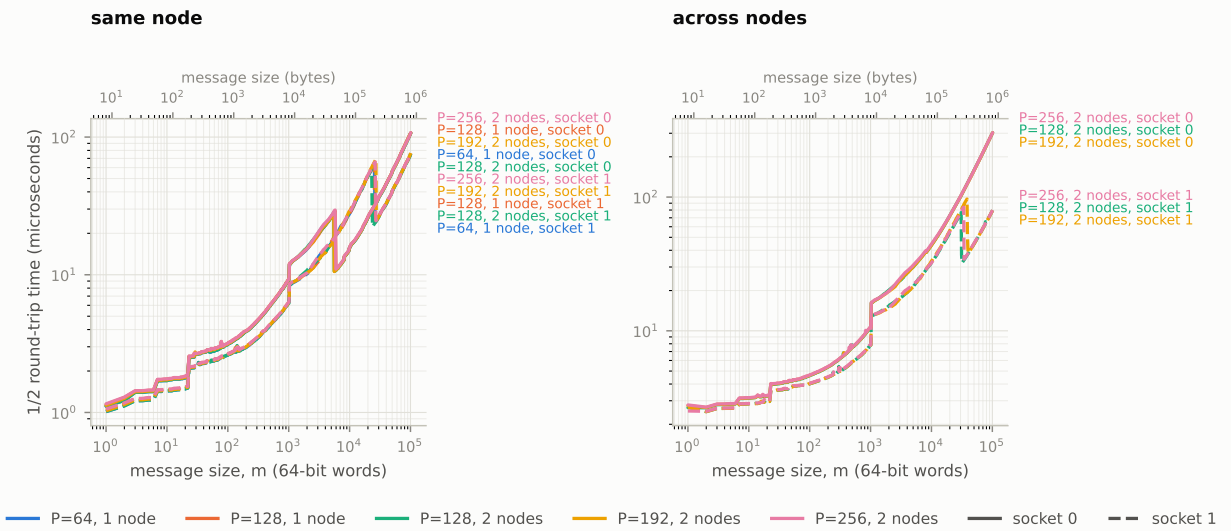


Figure 11: Per-socket median curves of all five configurations: same node (left) and across nodes (right).